Sir Syed University of Engineering & Technology, Karachi

CS-468
Software Quality Assurance & Testing

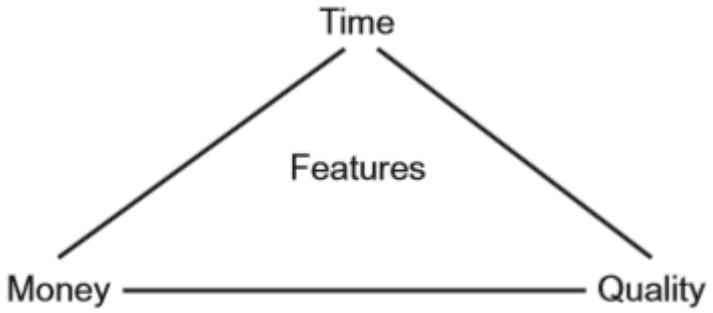
Week 5

Session 1

Testing

- Testing is an activity used to **reduce risk** and **improve quality** by finding **defects**.

Resources triangle



- One role for testing is to ensure that key **functional** and **non-functional** requirements are examined before the **system enters service** and **any defects are reported to the development team for rectification.**
- Testing cannot directly remove defects, nor can it directly enhance quality.
- By reporting defects it makes their removal possible and so contributes to the enhanced quality of the system.
- In addition, the systematic coverage of a software product in testing allows at least some aspects of the quality of the software to be measured.
- Testing is one component in the overall quality assurance activity that seeks to **ensure that systems enter service without defects that can lead to serious failures.**

Static and Dynamic Testing

- Static testing and dynamic testing **Static testing is the term used for testing where the code is not exercised.**
- Failures often begin with a human error, namely a mistake in a document such as a specification. We need to test these because errors are much cheaper to fix than defects or failures (as you will see).
- That is why testing should start as early as possible.
- Static testing involves techniques such as reviews, which can be effective in preventing defects, e.g. by removing ambiguities and errors from specification documents.
- **Dynamic testing is the kind that exercises the program under test with some test data.**
- The discipline of software testing encompasses both static and dynamic testing.

Retesting and Regression testing

- When anything changes (software, data, installation procedures, user documentation, etc.), we need to do two kinds of testing on the software:
- **Retesting:**
  - First of all, tests should be run to make sure that the problem has been fixed.
  - We are looking in fine detail at the changed area of functionality.

- **Regression testing:**
  - We also need to make sure that the changes have not broken the software elsewhere.
  - Should cover all the main functions to ensure that no unintended changes have occurred.

Effectiveness of the Tests

- Use well-proven **test design techniques**, and a selection of the most important of these.
- Use **principles of testing.**

Principles of testing

- **Testing shows the presence of bugs -** tests should be designed to find as many defects as possible.
- **Exhaustive testing is impossible** –
  - For large complex systems, exhaustive testing is not possible.
  - Unless the application under test (AUT) has an extremely simple logical structure and limited input, it is not possible to test all possible combinations of data input and circumstances.
  - **For this reason, risk and priorities are used to concentrate on the most important aspects to test.**
- **Early testing** –
  - The earlier a problem (defect) is found, the less it costs to fix.
  - A defect found at **acceptance testing** where the original mistake was in the requirements will require several work-products to be reworked, and subsequently to be retested.

Principles of testing

- **Defect clustering** –
  - The spread of defects is not uniform.
  - n a large application, it is often a small number of modules that exhibit the majority of the problems.
  - This can be for a variety of reasons, some of which are:
    - System complexity.
    - Volatile code.
    - The effects of change upon change.
    - Development staff experience.
    - Development staff inexperience.
  - This is the application of the Pareto principle to software testing:
    - ***approximately 80 per cent of the problems are found in about 20 per cent of the modules.***
  - **It is useful if testing activity reflects this spread of defects, and targets areas of the application under test where a high proportion of defects can be found.**

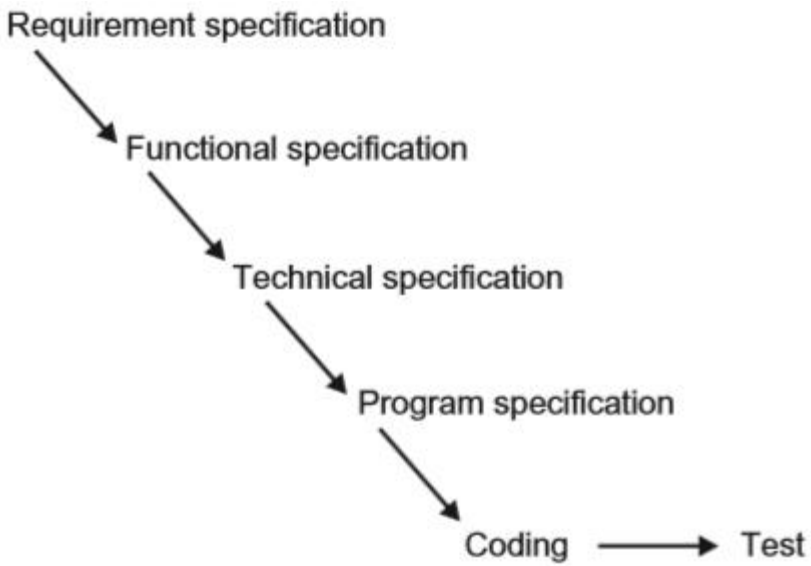
Principles of testing

- **The pesticide paradox** - Running the same set of tests continually will not continue to find new defects.
- **Testing is context dependent** - Different testing is necessary in different circumstances.
- **Absence of errors fallacy** - Software with no known errors is not necessarily ready to be shipped.

SDLC

- A **work-product** is an intermediate deliverable required to create the final product.
- Work-products can be documentation or code.
- The **code and associated documentation** will become the product when the system is declared ready for release.
- In software development, work-products are generally created in a series of defined stages, from capturing a customer requirement, to creating the system, to delivering the system.
- These stages are usually shown as steps within a software development life cycle.

Waterfall Model



- In the waterfall model, testing is carried out once the code has been fully developed.
- Once this is completed, a decision can be made on whether the product can be released into the live environment.
- This model for development shows how a fully tested product can be created, but it has a significant drawback**: what happens if the product fails the tests?**
- **In the waterfall model, the testing at the end serves as a quality check. The product can be accepted or rejected at this point.**

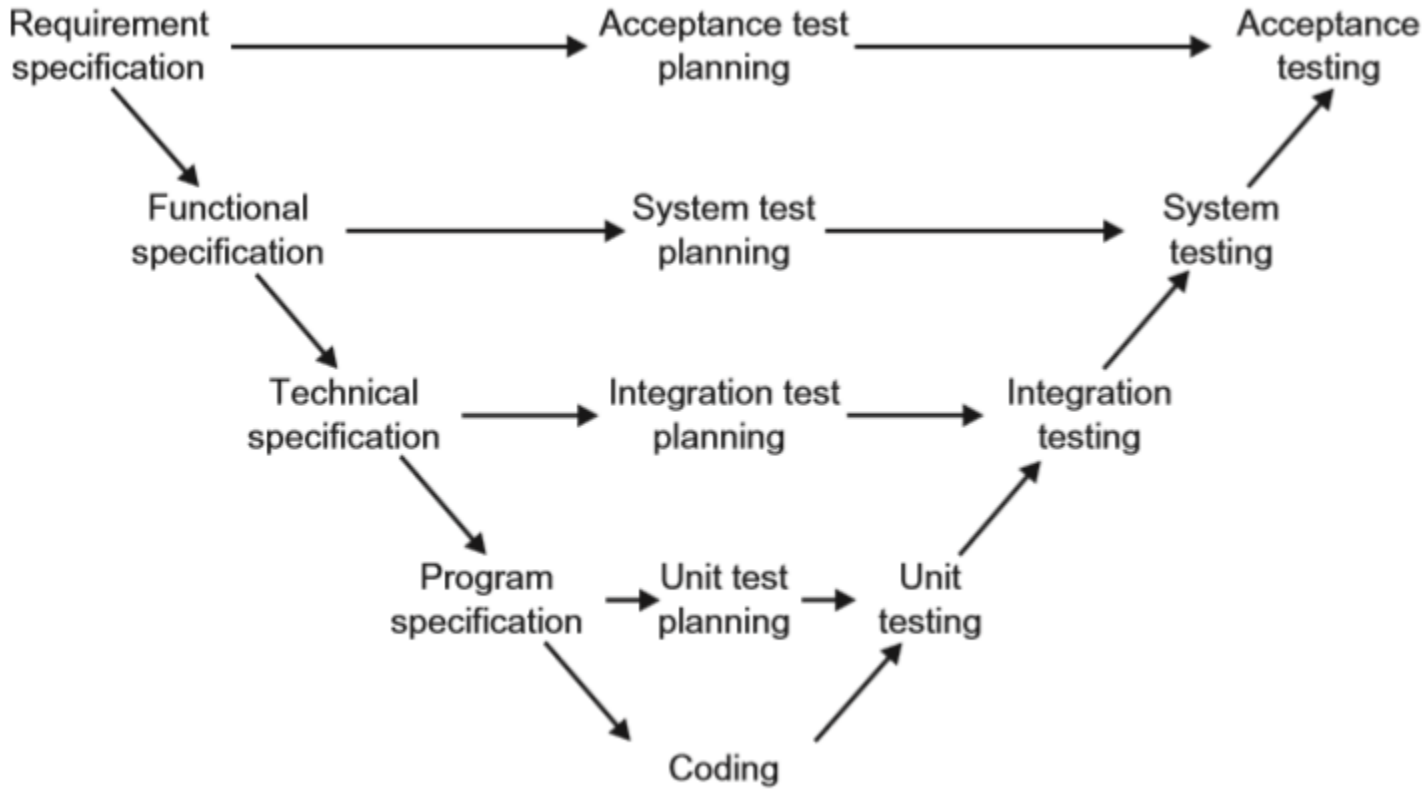
Remedy

- The checks throughout the life cycle include **verification and validation.**
- **Verification:**
  - **checks that the work-product meets the requirements set out for it.**
  - An example of this would be to ensure that a website being built follows the guidelines for making websites usable by as many people as possible.
  - **Verification helps to ensure that we are building the product in the right way.**
- **Validation:**
  - **changes the focus of work-product evaluation to evaluation against user needs.**
  - This means ensuring that the behaviour of the work-product matches the customer needs as defined for the project.
  - For example, for the same website above, the guidelines may have been written with people familiar with websites in mind.
  - It may be that this website is also intended for novice users. Validation would include these users checking that they too can use the website easily.
  - **Validation helps to ensure that we are building the right product as far as the users are concerned.**

Remedy

- There are two types of development model that facilitate early work-product evaluation.
- The first is an extension to the waterfall model, known as the V-model.
- The second is a cyclical model, where the coding stage often begins once the initial user needs have been captured. Cyclical models are often referred to as iterative models.
- We will consider the V-model.

The V Model


Requirement specification
Acceptance test planning
Acceptance testing
Functional specification
System test planning
System testing
Technical specification
Integration test planning
Integration testing
Program specification
Unit test planning
Unit testing
Coding

The V Model

- As for the waterfall model, the left-hand side of the model focuses on elaborating the initial requirements, providing successively more technical detail as the development progresses. In the model shown, these are:
  - **Requirement specification** – capturing of user needs.
  - **Functional specification** – definition of functions required to meet user needs.
  - **Technical specification** – technical design of functions identified in the functional specification.
  - **Program specification** – detailed design of each module or unit to be built to meet required functionality.
- These specifications could be reviewed to check for the following:
  - **Conformance to the previous work-product** (so in the case of the functional specification, verification would include a check against the requirement specification).
  - **That there is sufficient detail for the subsequent work-product to be built correctly** (again, for the functional specification, this would include a check that there is sufficient information in order to create the technical specification).
  - **That it is testable** – is the detail provided sufficient for testing the work-product?

The V Model

- The middle of the V-model shows that planning for testing should start with each work-product.
- Thus, using the requirement specification as an example, acceptance testing would be planned for, right at the start of the development.
- The right-hand side focuses on the testing activities. For each work-product, a testing activity is identified.
  - Testing against the requirement specification takes place at the acceptance testing stage.
  - Testing against the functional specification takes place at the system testing stage.
  - Testing against the technical specification takes place at the integration testing stage.
  - Testing against the program specification takes place at the unit testing stage.
- This allows testing to be concentrated on the detail provided in each work-product, **so that defects can be identified as early as possible in the life cycle, when the work-product has been created.**
- **Remembering that each stage must be completed before the next one can be started, this approach to software development pushes validation of the system by the user representatives right to the end of the life cycle.**
- If the customer needs were not captured accurately in the requirement specification, or if they change, then these issues may not be uncovered until the user testing is carried out.

Test Levels

- Testing helps to ensure that the work-products are being developed in **the right way (verification) and that the product will meet the user needs (validation).**
- Each work-product is tested – In the V-model, each document on the left is tested by an activity on the right. Each specification document is called the test basis, i.e. it is the basis on which tests are created. In iterative development, each release is tested before moving on to the next.
- Testers are involved in reviewing requirements before they are released – In the V-model, testers would be invited to review all documents from a testing perspective.
- Test stages on the right are often called test levels. The term test level provides an indication of the focus of the testing, and the types of problems it is likely to uncover. The typical levels of testing are:
  - **Unit (component) testing** - the focus is the code within the unit itself
  - **Integration testing** - it is the interfacing between units
  - **System testing** - it is the end-to-end functionality
  - **Acceptance testing** - it is the user perspective

Test Levels – Unit (Component) Testing …

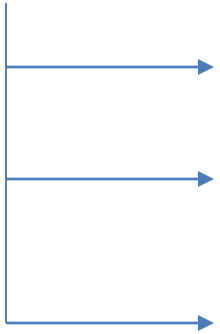
- Before testing of the code can start, clearly the code has to be written. This is shown at the bottom of the V-model.
- Generally, the code is written in component parts, or units. The units are usually constructed in isolation, for integration at a later stage.
- Units are also called programs, modules or components.
- **Unit testing is intended to ensure that the code written for the unit meets its specification, prior to its integration with other units.**
- In addition to checking conformance to the program specification, unit testing would also verify that all of the code that has been written for the unit can be executed.
- Instead of using the specification to decide on inputs and expected outputs, the developer would use the code that has been written for this.
- **Thus the test bases for unit testing can include: the component requirements; the detailed design; the code itself.**
- Unit testing requires access to the code being tested.
- Thus test objects (i.e. what is under test) can be the components, the programs, data conversion/migration programs and database modules.

Test Levels – Unit (Component) Testing

- Unit testing is often supported by a **unit test framework** (e.g. Kent Beck's Smalltalk Testing Framework: http:// xprogramming.com/testfram.htm). In addition, **debugging tools are often used**.
- An approach to unit testing is called **Test Driven Development**. As its name suggests, **test cases** are written first, code built, tested and changed until the unit passes its tests. This is an iterative approach to unit testing.
- Unit testing is usually performed by the developer who wrote the code (and who may also have written the program specification). **Defects found and fixed during unit testing are often not recorded.**

Test Levels – Integration Testing

- Once the units have been written, the next stage would be to put them together to create the system.
- This is called integration. It involves building something large from a number of smaller pieces.
- The purpose of integration testing is to expose defects in the interfaces and in the interactions between integrated components or systems.
- Thus the test bases for integration testing can include: the software and system design; a diagram of the system architecture; workflows and use-cases.
- The test objects would essentially be the interface code. This can include subsystems' database implementations.
- Before integration testing can be planned, an integration strategy is required. This involves making decisions on how the system will be put together prior to testing. There are three commonly quoted integration strategies, as follows.



Big Bang Integration - This is where all units are linked at once, resulting in a complete system.

Top down integration - This is where the system is built in stages, starting with components that call other components. Components that call others are usually placed above those that are called.

Bottom up integration - This is the opposite of top-down integration and the components are integrated in a bottom-up order.

Test Levels – Integration Testing …

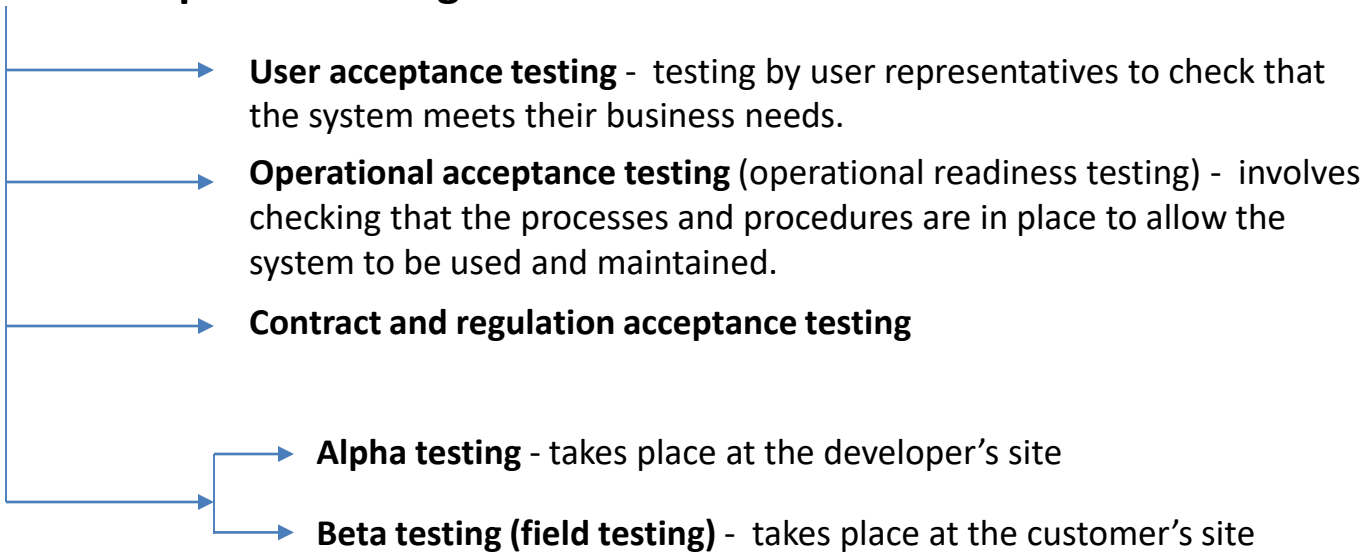
- There may be more than one level of integration testing. For example:
  - **Component integration testing**
    - Focuses on the interactions between software components and is done after component (unit) testing.
    - This type of integration testing is usually carried out by developers.
  - **System integration testing**
    - Focuses on the interactions between different systems and may be done after system testing of each individual system.
    - For example, a trading system in an investment bank will interact with the stock exchange to get the latest prices for its stocks and shares on the international market.
    - This type of integration testing is usually carried out by testers.
    - It should be noted that testing at system integration level carries extra elements of risk.
    - These can include: at a technical level, cross-platform issues; at an operational level, business workflow issues; and at a business level, risks associated with ownership of regression issues associated with change in one system possibly having a knock-on effect on other systems.

Test Levels – System Testing

- System testing is to consider the functionality from an end-to-end perspective.
- To test the system for functional and non functional requirements in detail.
- The amount of testing required at system testing, however, can be influenced by the amount of testing carried out (if any) at the previous stages.
- In addition, the amount of testing advisable would also depend on the amount of verification carried out on the requirements.
- Test bases for system testing can include: system and software requirement specifications; use cases; functional specifications; risk analysis reports; and system, user and operation manuals.
- The test object will generally be the system under test.

Test Levels – Acceptance Testing

- The purpose of acceptance testing is to provide the end users with confidence that the system will function according to their expectations.
- Referring once more to the V-model, acceptance testing will be carried out using the requirement specification as a basis for test.
- **The test bases can include:** user requirements; system requirements; use cases; business processes; and risk analysis reports.
- **The test objects can include:** the fully integrated system; forms and reports produced by the system.
- Acceptance testing may assess the system's readiness for deployment and use.
- Acceptance testing is often the responsibility of the customers or users of a system, although other project team members may be involved as well.
- **Forms of acceptance testing**:



Test Types

- Test types fall into the following categories:
  - **Functional testing** - looks at the specific functionality of a system,
  - **Non-functional testing** - This is where the behavioural aspects of the system are tested.
  - **Structural testing** - In structural testing, we change our measure to focus on the structural aspects of the system. This could be the code itself, or an architectural definition of the system.
  - **Testing after code has been changed** – retesting and regression testing

Maintenance Testing

- Maintenance testing is a testing required when a system has been released, but a change has become necessary
- There is a need for impact analysis in deciding how much regression testing to do after the changes have been implemented.

V model and Verification Validation

- The V-model typically has the following work-products and activities:
  (1) Requirement specification
  (2) Functional specification
  (3) Technical specification
  (4) Program specification
  (5) Code
  (6) Unit testing
  (7) Integration testing
  (8) System testing
  (9) Acceptance testing
- Work-products 1–5 would be subject to verification, to ensure that they have been created following the rules set out. For example, the program specification would be assessed to ensure that it meets the requirements set out in the technical specification, and that it contains sufficient detail for the code to be produced.
- In activities 6–9, the code is assessed progressively for compliance to user needs, as captured in the specifications for each level.